



OPEN SOURCE TOOLS FOR DATA SCIENCE

Project Title

APPSHIELD: Android Privacy and Malware Risk Analyzer

Programme	B.Sc. (Artificial Intelligence)
Academic Year	2026-27
Team ID	OST- <u>21</u>
Team Members	Srushti Lambdade - 25030421021
Faculty	Dr. Nabanita Choudhury, Assistant Professor

SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII)

Constituent of Symbiosis International (Deemed University)
Lavale, Pune - 412115

1. Abstract

Android apps ask for permissions such as reading messages or starting at boot. Some malware abuses these permissions, but users rarely check which ones an app asks for. APPSHIELD is a web application that estimates the malware risk of an Android app from the permissions and API calls it uses. I trained a Random Forest classifier on the TUANDROMD dataset, which has 4,464 apps and 241 features. The model reached 99.78% accuracy on a held-out test set of 893 apps. The application shows a risk level, a malware probability, and the indicators that pushed the result up or down. It also scores a CSV of many apps at once. The project uses open-source tools throughout: Python, scikit-learn, pandas, FastAPI and Uvicorn for the model and the web service, Docker Compose to run it, and Git and GitHub for version control. One command starts the whole system on any computer with Docker. I also explain the limits of the result: the dataset has many duplicate rows, so the accuracy is probably higher than it would be on apps from other sources. APPSHIELD is an educational prototype and not an antivirus product.

2. Introduction

Android is the most widely used mobile operating system, and people install apps from many sources. Every app must declare the permissions it needs, for example access to contacts, SMS or location. A normal app uses a small, sensible set. Malware often asks for combinations that normal apps do not.

Checking these permissions by hand is slow and needs knowledge most users do not have. A machine learning model can learn from labelled examples which permission patterns look like malware and give a quick estimate. A simple score alone is hard to trust, so we also wanted the tool to show which features led to its answer.

Open-source tools made this project possible at no cost. scikit-learn provides the model, pandas handles the data, FastAPI serves the application, and Docker packages everything so it runs the same way on every machine. Git and GitHub record the work and make it public.

3. Problem Statement and Objectives

Problem statement. Users and students have no simple, transparent way to estimate whether an Android app looks risky from its permissions. Existing tools often give a verdict without explaining it, and many research prototypes are hard to run on another computer.

Objectives

1. Train a classifier that separates malware from goodware using permission and API-call features.
2. Build a web application that scores one app and shows the reasons for the score.
3. Support scoring many apps at once from a CSV file.
4. Show model information such as the most important features and the class balance of the training data.
5. Package the whole project with Docker Compose so another person can run it with one command.
6. Use Git and GitHub with branches, pull requests and a README, and document the project clearly.
7. State the limits of the results honestly.

4. Dataset / Data Source

- **Name and source:** TUANDROMD (Tezpur University Android Malware Dataset), from the UCI Machine Learning Repository: <https://archive.ics.uci.edu/dataset/855/tuandromd>
- **Size:** 4,464 labelled apps (3,565 malware and 899 goodwill, so about 80% malware).
- **Fields:** 241 features. 214 are permission features and 27 are API-call features. The last column, Label, says Malware or Goodware.
- **Format:** a CSV file. Each feature is binary: 1 if the app uses it, 0 if not.
- **Licence:** CC BY 4.0.
- **Preprocessing.** I removed extra spaces from the column names, dropped rows with a missing label, converted the labels to numbers (Malware = 1, Goodware = 0) and replaced missing feature values with 0. The data was split 80% for training and 20% for testing, keeping the same class proportions (stratified split, random_state = 42). The CSV file has 4,465 rows; one row has no label and is removed, which leaves 4,464 apps.
- **Data check.** Only 660 of the 4,464 rows are distinct, so about 85% of the rows repeat another row's features. Only 2 distinct feature patterns appear with both labels.

5. Open-Source Technology Stack

Tool / Technology	Version (if applicable)	Purpose in the Project
Python	3.11 (Docker base image python:3.11-slim)	Main language for training and the web service
Scikit-learn	1.9.1	Random Forest model, train/test split, metrics
pandas	3.0.6	Loading and cleaning the CSV, batch scoring
Numpy	2.4.6	Building the feature vector for each app
joblib	1.6.0	Saving and loading the trained model
FastAPI	0.142.2	Web API that serves predictions and the interface
Uvicorn	0.54.0	Server that runs the FastAPI application
Python-multipart	0.0.32	Receiving the uploaded CSV in batch mode
HTML,CSS,Javascript	none	Single-page web interface, no build step

Docker and Docker Compose	Docker Desktop version 29.8.1 Docker compose version v5.5.1	Packaging and running the trainer and the web app
Git and GitHub	2.55.0.windows.2	Version control, branches, pull requests
Mermaid	none (built into GitHub)	Architecture diagram in the README

6. System Architecture / Workflow



Figure : APPSHIELD architecture with two Docker Compose services.

The system has two Docker services.

1. The **trainer** service reads data/TUANDROMD.csv, trains the Random Forest, prints the evaluation, saves model/model.pkl and then exits.
2. The **appshield** service starts after the trainer finishes. It loads the model and serves both the API and the web interface on port 8501.
3. The user opens <http://localhost:8501> in a browser.

Workflow:

“Inside the application, an analysis follows four steps. The Profile is the set of selected permissions and API calls. The Vector turns them into a list of 241 values (0 or 1). The Random Forest of 200 trees then votes malware or goodware. The Diagnostic shows the share of trees voting malware as the malware probability, together with the supporting indicators.”

7. Implementation

Data loading and preprocessing. train_model.py reads the CSV with pandas and applies the cleaning steps in Section 4.

Model development. I used RandomForestClassifier with 200 trees and a fixed random_state so results can be repeated. After training, the script prints accuracy and a classification report. It then picks one real malware example and one real goodware example from the test set. These become the sample buttons in the interface. The model, the feature list, the samples and the test accuracy are saved together in model.pkl.

Web service (server.py). It loads the saved model once at startup and provides four endpoints:

- GET /api/meta: model details, feature list, samples and the most important features
- POST /api/analyze: scores one app

- POST /api/batch: scores an uploaded CSV
- GET /api/template: downloads a CSV template with the correct column names

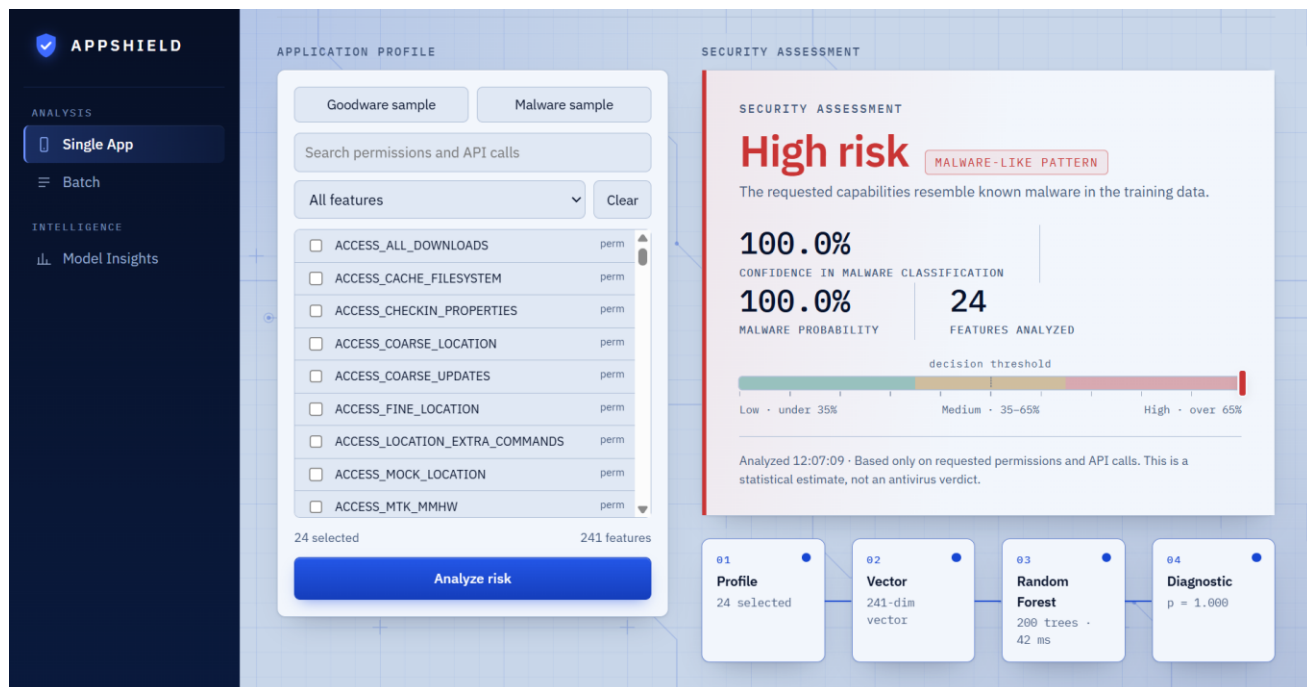
Explaining a result. For each selected feature, the service removes it from the app's profile, scores the app again, and reports the change in malware probability. A large positive change means that feature pushed the score toward malware. This is a simple "what if this feature were missing" method that uses the real model.

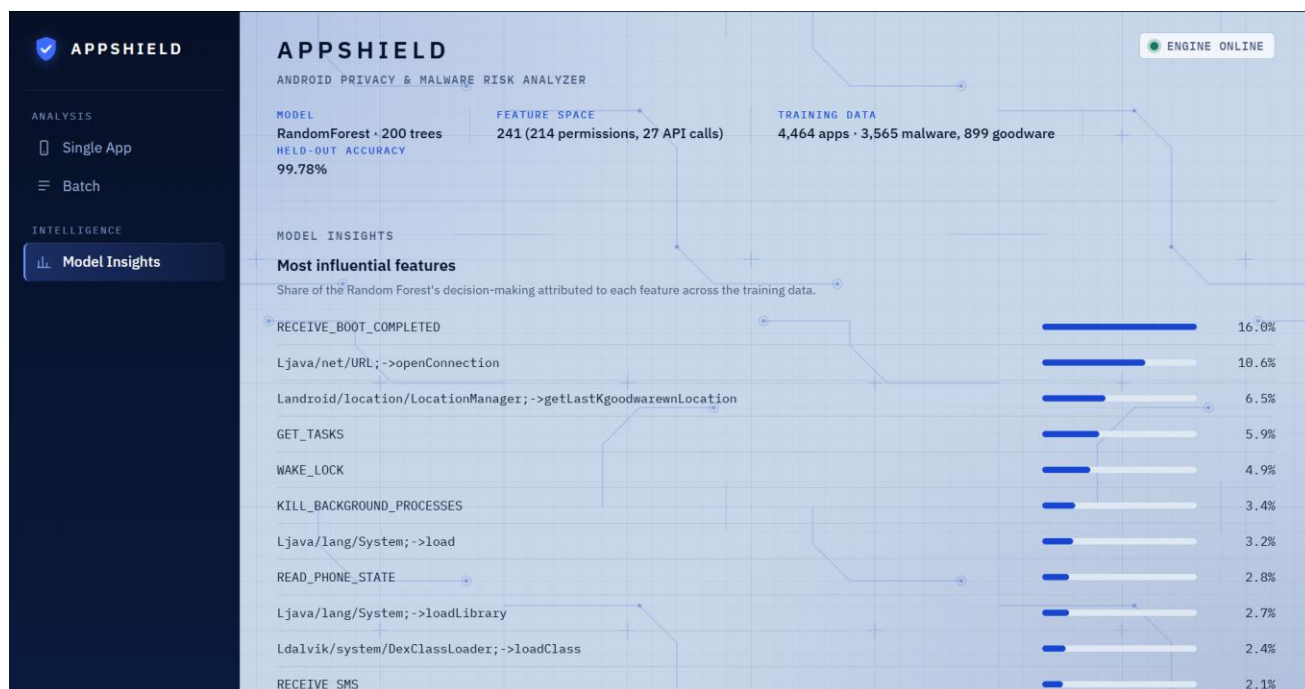
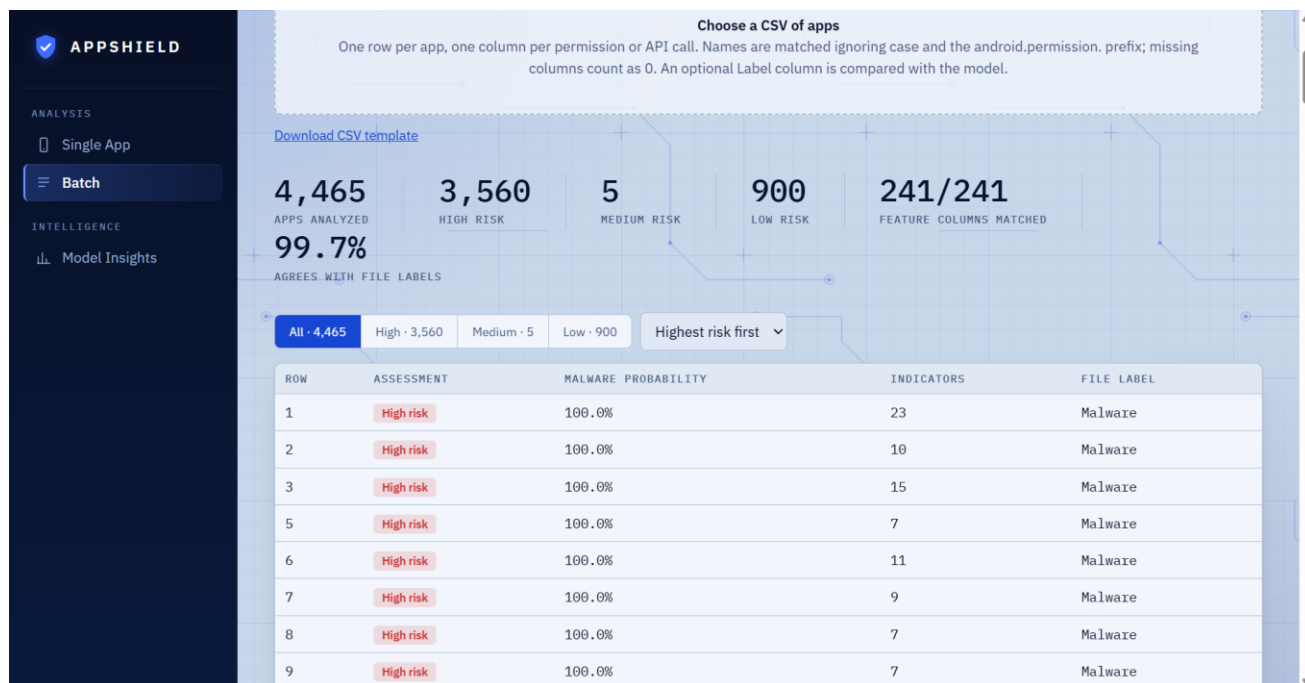
Risk levels. The model decides malware or goodware at 50% probability. For display I group the probability into Low (under 35%), Medium (35% to 65%) and High (over 65%). These cut-offs are our own choice and the interface states them.

Batch mode. The service matches CSV columns to the model's features while ignoring capital letters and the prefix android.permission.. Missing columns count as 0. If the file has a Label column, the page also reports how often the model agrees with it.

Interface (static/index.html). A single HTML file with CSS and JavaScript. It has three pages (Single App, Batch, Model Insights), a light and a dark theme, and a layout that adapts to phones and tablets. The security result is the main element on the page, followed by the analysis pipeline.

Relevant Screenshots.





8. GitHub Repository and Version Control

- Repository: <https://github.com/lambdaderushti/appshield> (public)
- Structure: data/, model/, static/, docs/, server.py, train_model.py, Dockerfile, docker-compose.yml, requirements.txt, README.md, LICENSE.
- Commit history: 20 commits so far, with messages describing each step, such as moving to the FastAPI backend and adding screenshots.
- Branches: main plus two feature branches, feature/docs-and-screenshots and feature/ca3-feedback-and-screenshots. Each was merged into main through a pull request (PR #1 and PR #2).
- Tag: v1.0 marks the final submitted version.
- Collaboration: The project was carried out individually, so all commits are by one author.

The screenshot shows the GitHub repository page for `lambdaderushti/appshield`. The repository is public and has 20 commits. The commit history is as follows:

File	Commit Message	Time
data	Switch to FastAPI backend with custom frontend	2 days ago
docs	Add screenshots, reproducibility note and CA3 feedback	11 hours ago
model	Add screenshots, reproducibility note and CA3 feedback	11 hours ago
static	Add complete APPSHIELD FastAPI project	yesterday
.gitignore	Add complete APPSHIELD FastAPI project	yesterday
Dockerfile	Add complete APPSHIELD FastAPI project	yesterday
LICENSE	Add complete APPSHIELD FastAPI project	yesterday
README.md	Add screenshots, reproducibility note and CA3 feedback	11 hours ago
docker-compose.yml	Add complete APPSHIELD FastAPI project	yesterday
requirements.txt	Add complete APPSHIELD FastAPI project	yesterday
server.py	Add complete APPSHIELD FastAPI project	yesterday
train_model.py	Add complete APPSHIELD FastAPI project	yesterday

The repository also includes a README, a MIT license, and a Dockerfile. The repository is tagged with v1.0. The repository statistics show 0 stars, 0 forks, and 0 watchers.

Add screenshots, reproducibility note and CA3 feedback #2

Merged lambdaderushti merged 1 commit into main from feature/ca3-feedback-and-screenshots 18 hours ago

Conversation 0 Commits 1 Checks 0 Files changed 6

lambdaderushti commented 18 hours ago

No description provided.

Add screenshots, reproducibility note and CA3 feedback

lambdaderushti merged commit b18c218 into main 18 hours ago

Pull request successfully merged and closed

You're all set — the feature/ca3-feedback-and-screenshots branch can be safely deleted.

Add a comment

Write Preview

Add your comment here...

Reviewers
No reviews

Assignees
No one—assign yourself

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request may close these issues.
None yet

Notifications
Unsubscribe Customize

Add architecture diagram to README #1

Merged lambdaderushti merged 1 commit into main from feature/docs-and-screenshots 19 hours ago

Conversation 0 Commits 1 Checks 0 Files changed 1

lambdaderushti commented 19 hours ago

No description provided.

Add architecture diagram to README

lambdaderushti merged commit 9384684 into main 19 hours ago

Pull request successfully merged and closed

You're all set — the feature/docs-and-screenshots branch can be safely deleted.

Add a comment

Write Preview

Add your comment here...

Reviewers
No reviews

Assignees
No one—assign yourself

Labels
None yet

Projects
None yet

Milestone
No milestone

Development
Successfully merging this pull request may close these issues.
None yet

Notifications
Unsubscribe Customize

9. Reproducibility and Environment Setup

Requirements: Docker Desktop (it includes Docker Compose). Python is only needed for the optional non-Docker route.

Dependency file: requirements.txt lists pandas, numpy, scikit-learn, joblib, fastapi, uvicorn and python-multipart.

Steps:

git clone https://github.com/lambdadesrushti/appshield.git
cd appshield
docker compose up --build

Then open <http://localhost:8501>. The page shows **Engine online** when it is ready.

Without Docker: pip install -r requirements.txt, then python train_model.py, then uvicorn server:app --port 8501.

Verification. I have tested the setup from a fresh clone in an empty folder on Windows 11 with Docker Desktop (Figure 6, the terminal output). The first build takes several minutes because packages are downloaded. Later runs are much faster because Docker reuses what it already built.

Configuration: the two Docker files below. The application needs no separate environment variables.

10. Containerization / Deployment

Dockerfile. It starts from python:3.11-slim, copies requirements.txt, installs the packages, copies the project and sets the command to start Uvicorn on port 8501. Copying the requirements file first lets Docker reuse the installed packages when only the code changes.

docker-compose.yml. It defines two services built from the same image:

- trainer runs python train_model.py and exits.
- appshield runs the web service, publishes port 8501, and waits until the trainer has finished successfully (depends_on with service_completed_successfully).

The data/ and model/ folders are shared with the containers as volumes, so the trained model saved by the trainer is the one the web service loads.

Build and run: docker compose up --build. To stop: Ctrl+C, then docker compose down.

Deployment. I ran the project locally in containers. It has not been deployed to a cloud host. Because it is a standard Docker project, it could be deployed to any host that runs containers.

11. Results and Output

Model evaluation (20% held-out test set, 893 apps, as printed by the trainer):

Class	Precision	Recall	F1-score	Test apps
-------	-----------	--------	----------	-----------

Goodware	1.00	0.99	0.99	180
----------	------	------	------	-----

Malware	1.00	1.00	1.00	713
---------	------	------	------	-----

Application output. In the demo, the malware sample is rated High risk with a malware probability close to 100%. The goodware sample is rated Low risk with a probability close to 0%. For the malware sample, the evidence panel showed RECEIVE_BOOT_COMPLETED, GET_TASKS and WAKE_LOCK as the indicators that raised the score most.

Batch result. Scoring the whole dataset gave about 900 Low, 5 Medium and 3,560 High, and the model agreed with the dataset's own labels about 99.7% of the time. This is not a fair test, because the model was trained on these same apps.

What the output shows. The permission pattern of an app carries a strong signal for this dataset. The interface also makes the reasoning visible instead of only giving a label.

Interpreting the accuracy. The very high score is partly because of the duplicate rows described in Section 4: most test apps have an identical copy in the training data. The dataset is also about 80% malware, so a model that always answered "malware" would already be right about 80% of the time. The score therefore shows the model fits this dataset well, not that it would be this accurate on new apps from elsewhere.

12. Open-Source Licensing and Dependencies

Resource	Licence
This project (APPSHIELD)	MIT
TUANDROMD dataset	CC BY 4.0 (credit the authors)
scikit-learn, pandas, NumPy, joblib	BSD 3-Clause
FastAPI, Pydantic	MIT
Uvicorn, Starlette	BSD 3-Clause
python-multipart	Apache 2.0
Python	PSF licence

Docker Engine / Compose	Apache 2.0
Docker Desktop	Docker's own terms; free for education and personal use
Git	GPL v2
IBM Plex fonts (loaded from Google Fonts)	SIL Open Font Licence 1.1
Mermaid	MIT

13. Challenges Faced and Solutions

1. **Replacing the first interface.** The original Streamlit dashboard limited how much I could change the design, and its pipeline text described steps that did not really happen. I moved to a FastAPI backend with a custom interface and kept only steps that the program really performs.
2. **All batch results looked "High risk".** The table showed only the 200 highest-risk apps, which are all malware. The summary counts were correct. I changed the table to show every app, with filters by risk level and sorting.
3. **Batch uploads of other CSV files failed.** The model needs the exact feature columns. I made column matching more forgiving, added a CSV template and gave a clear error message when no columns match.
4. **Browser error ERR_ADDRESS_INVALID.** The server prints 0.0.0.0:8501, which a browser cannot open. The correct address is localhost:8501. This is now in the README.
5. **Git pushed to the wrong place.** My project folder sat inside a Git repository in the home folder that pointed to a different project, so pushing was refused. I made the project folder its own repository linked to the correct GitHub repository.
6. **Model file version warning.** A model saved with one scikit-learn version gave a warning on another. Running training and serving inside the same Docker image removes this problem.
7. **Docker not starting.** A "cannot connect to the Docker daemon" error appeared because Docker Desktop was not running. Starting it first solves this.
8. **Unexpectedly high accuracy.** I investigated, found the duplicate rows, and documented the limitation instead of presenting the score as proof.

14. Limitations

- The model understands only the 241 TUANDROMD features. Files with other columns cannot be scored.
- The app does not read APK files. A person must select the permissions or upload a CSV.
- The dataset is imbalanced (about 80% malware).

- Only 660 distinct feature patterns exist in the data, so the 99.78% score is optimistic and untested on apps from other sources.
- Features are only yes/no. The model sees no code behaviour, network activity or app reputation.
- The Low, Medium and High cut-offs are our own choice and have not been calibrated.
- The explanation is a simple remove-one-feature check and does not capture feature combinations.
- It is a prototype for learning and is not an antivirus.
- The package versions in requirements.txt are not pinned.

15. Conclusion and Future Scope

Conclusion. I built APPSHIELD, a containerised web application that estimates the malware risk of an Android app from its permissions and API calls. The Random Forest reached 99.78% accuracy on the held-out test set, and the interface explains each result with the features that influenced it. Using open-source tools, I went from a raw dataset to a working application that runs with one Docker Compose command, and I practised branching, pull requests and documentation on GitHub. I also learned that a very high score needs checking, because duplicate data inflated ours.

Future scope

- Remove duplicate rows before splitting the data and report that score too.
- Read permissions directly from an APK file or manifest.
- Test on a second dataset and compare.
- Add stronger explanation methods such as SHAP.
- Pin dependency versions and add automated tests and a GitHub Actions workflow.
- Calibrate the risk levels and keep a history of past analyses.
- Deploy it on a container hosting service.

16. References

- UCI Machine Learning Repository. (n.d.). *TUANDROMD dataset*. <https://archive.ics.uci.edu/dataset/855/tuandromd>
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
- Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- FastAPI. (n.d.). *FastAPI documentation*. <https://fastapi.tiangolo.com>
- Docker. (n.d.). *Docker Compose documentation*. <https://docs.docker.com/compose/>
- GitHub. (n.d.). *GitHub Docs*. <https://docs.github.com>
- Mermaid. (n.d.). *Mermaid documentation*. <https://mermaid.js.org>

17. Team Contribution

Clearly state the specific contribution of each team member.

Team Member	PRN	Specific Contribution
Srushti Lambdade	25030421021	This project was completed individually. All work, including dataset analysis, model training, backend and interface development, containerisation, version control and documentation, was done by me.

18. Annexure

```
=> => naming to docker.io/library/appshield-appshield:latest 0.0s
=> => unpacking to docker.io/library/appshield-appshield:latest 0.2s
=> [trainer] exporting to image 1.0s
trainer-1 | Training Random Forest model...
trainer-1 |
trainer-1 | Test accuracy: 0.9978
trainer-1 |
trainer-1 |           precision    recall  f1-score   support
trainer-1 |
trainer-1 |      Goodware         1.00      0.99      0.99        180
trainer-1 |      Malware          1.00      1.00      1.00        713
trainer-1 |
trainer-1 |      accuracy
trainer-1 |      macro avg         1.00      0.99      1.00        893
trainer-1 |      weighted avg      1.00      1.00      1.00        893
trainer-1 |
trainer-1 | Sample malware example: 24 permissions selected
trainer-1 | Sample goodware example: 9 permissions selected
trainer-1 | Model saved to model/model.pkl
Container appshield-trainer-1 Exited
trainer-1 exited with code 0
appshield-1 | INFO: Started server process [1]
appshield-1 | INFO: Waiting for application startup.
appshield-1 | INFO: Application startup complete.
appshield-1 | INFO: Uvicorn running on http://0.0.0.0:8501 (Press CTRL+C to quit)
appshield-1 | INFO: 172.18.0.1:60026 - "GET / HTTP/1.1" 200 OK
appshield-1 | INFO: 172.18.0.1:60026 - "GET /api/meta HTTP/1.1" 200 OK
```

lambdadesrushti / appshield

<> Code

Issues

Pull requests

Actions

Projects

Security and quality

Insights

Settings

Q Type to search

+

main

All users

All time

Commits on Oct 8, 2026

Merge pull request #2 from lambdadesrushti/feature/ca3-feedback-and-screenshots

Verifiedb18c218

lambdadesrushti authored 12 hours ago

Add screenshots, reproducibility note and CA3 feedback

902923f

lambdadesrushti committed 12 hours ago

Merge pull request #1 from lambdadesrushti/feature/docs-and-screenshots

Verified9384684

lambdadesrushti authored 12 hours ago

Add architecture diagram to README

124aa0f

lambdadesrushti committed 13 hours ago

Commits on Oct 7, 2026

Update README

114b69d

lambdadesrushti committed yesterday

Add complete APPSHIELD FastAPI project

b8a6701

lambdadesrushti committed 2 days ago

Switch to FastAPI backend with custom frontend

8691c26

lambdadesrushti committed 2 days ago

Commits on Oct 5, 2026

lambdadesrushti committed < 4 days ago

Commits on Oct 5, 2026

Deploy pristine native Streamlit layout, remove buggy HTML containers

92e9ae8

lambdadesrushti committed 3 days ago

Fix Red Theme Issue and Implement High-Contrast Developer Console

5791e80

lambdadesrushti committed 3 days ago

Fix Layout & Background - Remove Graph Paper Grid and HTML wrapper conflict

23a350f

lambdadesrushti committed 3 days ago

Final Deploy: Premium Technical Blue Product UI with Result-First Architecture

3859011

lambdadesrushti committed 3 days ago

Deploy Final Premium UI: Technical Blue, Animated Pipeline, and Live Log Feed

37f2d23

lambdadesrushti committed 4 days ago

Restore sidebar, pages, and fix red tags

34788f8

lambdadesrushti committed 4 days ago

Final Premium Redesign - Technical Blue Aesthetic with Pipeline Visualization

d5e7a7b

lambdadesrushti committed 4 days ago

Deploy Split-Pane Enterprise UI and Dark Theme Config

a8f7204

lambdadesrushti committed 4 days ago

Deploy Vercel-style enterprise UI

742eeea

lambdadesrushti committed 4 days ago

Commits on Oct 4, 2026

Apply vibrant dark neon cyber theme

4178e10

lambdadesrushti committed 4 days ago

Update dashboard UI

b5b0588

lambdadesrushti committed 4 days ago

